🌍 MANGO — Phase 1–10 Documentation

Meteorological AI Network for Geospatial Operations

AI Powered Digital Twin of India's Climate

🟦 Phase 1 — Project Setup & Team Initialization

Build the foundation for development

Create the repository structure, configure collaboration workflows, assign responsibilities, establish branch management, and prepare the development environment to ensure smooth execution throughout the project lifecycle.

🟩 Phase 2 — Data Collection & Organization

Acquire and structure climate intelligence sources

Collect and organize national climate datasets from IMD and INSAT, validate data integrity, establish storage conventions, and prepare the raw data layer required for climate modeling.

🟧 Phase 3 — Data Processing & Feature Engineering

Transform raw observations into model-ready information

Clean, align, normalize, and integrate climate datasets while generating meaningful temporal and spatial features suitable for machine learning and simulation.

🟧 Phase 4 — Exploratory Data Analysis (EDA)

Understand climate behaviour before modeling

Analyze rainfall and temperature patterns, detect trends and anomalies, visualize distributions, and derive insights that guide model selection and feature design.

🟥 Phase 5 — Baseline Machine Learning Model

Build the first working forecasting engine

Develop initial predictive models for rainfall and temperature using classical machine learning approaches to establish benchmark performance and validate feasibility.

🟪 Phase 6 — Advanced Forecasting Model

Improve prediction using temporal learning

Implement advanced sequence-based models such as LSTM to capture climate dynamics and improve forecasting accuracy over baseline approaches.

🟫 Phase 7 — Digital Twin Engine Development

Convert predictions into an interactive climate replica

Build the digital twin layer that maintains current climate state, performs forecasting, and enables scenario-based simulations through a unified backend system.

🟦 Phase 8 — Dashboard & Visualization Development

Transform outputs into an interactive user experience

Develop a dynamic frontend dashboard featuring maps, forecasts, simulations, and animated visualizations for intuitive exploration of climate intelligence.

🟪 Phase 9 — System Integration & Validation

Combine all components into one complete platform

Integrate frontend, backend, AI models, and datasets into a unified application while validating performance, reliability, and user interaction flows.

🟧 Phase 10 — Finalization, Presentation & Submission

Prepare the solution for evaluation and demonstration

Polish the platform, prepare technical documentation, create presentation materials, conduct demonstrations, and package the final submission for the hackathon.

🚀 Project Flow

Setup
↓

Data Collection
↓

Processing
↓

EDA
↓

Baseline ML
↓

Advanced ML
↓

Digital Twin
↓

Dashboard
↓

Integration
↓

Submission

🟦 PHASE 1 — PROJECT SETUP & TEAM INITIALIZATION

🎯 Objective

Create the engineering foundation of the project before writing any ML or frontend code.

At the end of this phase:

✅ Repository exists
✅ Team members connected
✅ Folder structure finalized
✅ Branch workflow established
✅ Documentation started

Duration:

🕑 **1–2 Days**

📌 Step 1.1 — Create GitHub Repository

Repository Name:

mango-climate-twin

Description:

AI-powered Digital Twin of India's Climate using national datasets.

Visibility:

🟢 Public

Initialize:

✓ README
✓ .gitignore → Python

Result:

GitHub Repo Ready

📌 Step 1.2 — Documentation Setup

Inside docs:

Create:

docs/

architecture.md

timeline.md

meeting_notes.md

📌 Step 1.3 — Define Roles

🟢 Data Engineer

Owns:

data/

🔵 ML Engineer

Owns:

ml/

🟠 Backend Engineer

Owns:

backend/

🟣 Frontend Engineer

Owns:

frontend/

✅ Deliverables (Phase 1)

Repo

Branches

Structure

Roles

Documentation

🟩 PHASE 2 — DATA COLLECTION

🎯 Objective

Build the **data source layer** of the Digital Twin.

Climate Twin depends entirely on data quality.

Duration:

🕒 **2–4 Days**

🌧 Step 2.1 — Identify Required Datasets

Collect:

Dataset	Purpose
IMD Rainfall	Ground Truth
IMD Max Temp	Surface Temperature
IMD Min Temp	Surface Temperature
INSAT Rainfall	Satellite Observation
INSAT SST	Ocean Influence
INSAT LST	Surface Heat

📁 Step 2.2 — Organize Storage

Inside:

data/

Create:

raw/

imd/

insat/

Result:

data/

raw/

imd/

insat/

📌 Step 2.3 — Define Naming Convention

Good:

rainfall_2015.csv

max_temp_2015.csv

Bad:

data1.csv

new.csv

Rule:

dataset_year_version

📌 Step 2.4 — Create Metadata File

Create:

data_metadata.xlsx

Columns:

| Dataset |
 | Source |
 | Date |
 | Resolution |
 | Unit |

Example:

Rainfall

IMD

0.25°

📌 Step 2.5 — Validate Downloads

Verify:

🟢 File opens
🟢 No corruption
🟢 Correct years

Check:

rows
columns
missing

📌 Step 2.6 — Exploratory Inspection

Open sample data.

Inspect:

date

latitude

longitude

rain

temperature

Write observations.

Example:

Rainfall missing after 2019

📌 Step 2.7 — Dataset Summary

Document:

Time Range

Resolution

Coverage

Units

✅ Deliverables (Phase 2)

Raw datasets

Metadata

Directory

Validation report

🟧 PHASE 3 — DATA PROCESSING & FEATURE PIPELINE

🎯 Objective

Convert raw climate files into ML-ready structured data.

Duration:

🕑 **3–5 Days**

🔄 Step 3.1 — Create Pipeline

Flow:

Raw

↓

Cleaning

↓

Transformation

↓

Features

↓

Processed

🧹 Step 3.2 — Data Cleaning

Tasks:

Missing Values

Methods:

mean

interpolation

Remove Duplicates

Example:

same region

same day

Standardize Units

Example:

mm

°C

🗓️ Step 3.3 — Time Alignment

Align:

Rain

Temp

Satellite

Example:

2020-06-10

must exist across datasets.

🌍 Step 3.4 — Spatial Alignment

Convert:

Different grids

↓

Single grid

Example:

0.25°

🧠 Step 3.5 — Feature Engineering

Create:

Feature

Rain_t−1

Rain_t−7

Temp_t−1

Month

Season

Example:

Rain_today

↓

Predict

↓

Rain_tomorrow

📊 Step 3.6 — Normalize Data

Methods:

MinMax

StandardScaler

Result:

0–1

range.

💾 Step 3.7 — Export Processed Dataset

Save:

data/

processed/

climate_ready.csv

Include:

feature_info.json

📌 Step 3.8 — Create Data Report

Document:

Metric
Samples
Features
Missing
Resolution

✅ Deliverables (Phase 3)

Clean Dataset

Features

Processed CSV

Pipeline Script

🟥 PHASE 4 — EXPLORATORY DATA ANALYSIS (EDA)

🎯 Objective

Understand climate behaviour before training models.

This phase answers:

🌧 How rainfall behaves
🌡 Temperature trends
📈 Seasonal patterns
⚠ Extreme events
🦟 Data quality issues

Duration:

🕐 **2–4 Days**

🧭 Why EDA Matters

Bad data →

Bad Model

Good EDA →

Better Forecast

📁 Inputs

Use:

data/processed/

Example:

climate_ready.csv

Columns:

date
lat
lon
rain
temp_max
temp_min

📌 Step 4.1 — Dataset Summary

Calculate:

Metric
Rows
Columns
Missing
Time Range

Output:

eda_summary.md

📊 Step 4.2 — Distribution Analysis

Visualize:

Rainfall Distribution

Questions:

Is rainfall skewed?

Extreme values?

Temperature Distribution

Check:

normal?

seasonal?

Charts:

📈 Histogram
📦 Boxplot

🌧 Step 4.3 — Time Series Analysis

Plot:

Date

↓

Rainfall

Observe:

🟢 Monsoon peaks
🟢 Dry periods
🟢 Anomalies

🌡 Step 4.4 — Temperature Trends

Create:

Monthly Average

Look for:

warming

seasonality

🌍 Step 4.5 — Spatial Visualization

Generate maps:

🗺 Rainfall Heatmap
🗺 Temperature Heatmap

Questions:

Which regions differ?

🔗 Step 4.6 — Correlation Analysis

Compare:

Rain

Temp

Month

Output:

📌 Correlation Matrix

Goal:

Find important predictors.

⚠ Step 4.7 — Outlier Detection

Detect:

extreme rainfall

unusual temperature

Handle:

remove

clip

keep

📄 Deliverables (Phase 4)

EDA Notebook

Charts

Heatmaps

Feature Report

🟥 PHASE 5 — BASELINE ML MODEL

🎯 Objective

Build the **first working prediction model**.

Goal:

Predict:

🌧 Rainfall
🌡 Temperature

Duration:

🕒 **4–5 Days**

🧠 Why Baseline First?

Never jump directly into deep learning.

Pipeline:

Simple

↓

Working

↓

Improve

📁 Inputs

Use:

processed dataset

📌 Step 5.1 — Define Target

Example:

Inputs:

Rain_t−1

Rain_t−7

Month

Temp

Output:

Rain_t

📌 Step 5.2 — Train/Test Split

Recommended:

80%

20%

Important:

❌ No random shuffle for time series.

Use:

chronological split

📌 Step 5.3 — Baseline Model Selection

Order:

🥇 Linear Regression

↓

🥈 Random Forest

↓

🥉 XGBoost

📌 Step 5.4 — Training

Pipeline:

Load

↓

Train

↓

Predict

Save:

model.pkl

📌 Step 5.5 — Evaluation

Metrics:

MAE

average error

RMSE

penalizes large errors

R^2

fit quality

📌 Step 5.6 — Error Analysis

Visualize:

Actual

↓

Predicted

Questions:

Underfitting?

Overfitting?

📄 Deliverables (Phase 5)

baseline model

metrics

graphs

🟪 PHASE 6 — ADVANCED MODEL (TIME SERIES + DL)

🎯 Objective

Improve prediction quality using sequence learning.

Duration:

🕒 **5–7 Days**

🧠 Model Choice

Recommended:

🥇 LSTM

Optional:

🥈 ConvLSTM

Skip:

❌ Transformers initially

📌 Step 6.1 — Convert to Sequences

Example:

Input:

last 7 days

Predict:

next day

Sequence:

Day1

Day2

Day3

↓

Day8

📌 Step 6.2 — Model Architecture

Structure:

Input

↓

LSTM

↓

Dense

↓

Output

📌 Step 6.3 — Training Strategy

Epochs:

20–50

Batch:

32

Loss:

MSE

Optimizer:

Adam

📌 Step 6.4 — Hyperparameter Tuning

Tune:

window

hidden size

learning rate

Keep notes.

📌 Step 6.5 — Compare Models

Create table:

Model MAE RMSE

Goal:

Select final model.

📌 Step 6.6 — Export Final Model

Save:

models/

forecast.pt

📄 Deliverables (Phase 6)

trained model

comparison

weights

🟫 PHASE 7 — DIGITAL TWIN ENGINE

🎯 Objective

Turn prediction into a living climate system.

Duration:

🕒 **5–6 Days**

🧠 What is Digital Twin Here?

Not:

just ML

But:

Current State

+

Prediction

+

Simulation

🏗 Architecture

Data

↓

Model

↓

Twin Engine

↓

API

↓

Dashboard

📌 Step 7.1 — Current Climate State

System computes:

today rainfall

today temperature

Endpoint:

/current

Output:

JSON

📌 Step 7.2 — Forecast Engine

Input:

region

Output:

next day

Endpoint:

/forecast

🟫 Add: Phase 7.3 — GenAI-Powered Scenario Simulation

🎯 Objective

Allow users to describe climate scenarios in natural language and generate simulations automatically.

Instead of users manually changing sliders:

Temp +2°C

Rain −10%

users type:

> “Show what happens if monsoon rainfall decreases and heat increases in central India.”

The system interprets this and simulates.

🧠 System Idea

Current:

User

↓

Manual Inputs

↓

Simulation

New:

User Prompt

↓

GenAI

↓

Scenario Parameters

↓

Digital Twin

↓

Prediction

↓

Generated Insight

📌 Step 7.3.1 — Natural Language Scenario Input

Frontend:

Create:

Scenario Input Box

Examples:

Increase temperature by 3°C.

Reduce rainfall by 20%.

Simulate severe drought.

Simulate delayed monsoon.

📌 Step 7.3.2 — Prompt Interpreter (LLM Layer)

GenAI converts text →

structured parameters.

Example:

User:

Simulate delayed monsoon and hotter conditions.

Generated:


```
{
"temp_change":2,
"rain_shift":-15,
"monsoon_delay":10
}
```

📌 Step 7.3.3 — Scenario Engine

Backend receives:

temperature offset

rainfall offset

Twin modifies state.

Example:

Original:

32°C

40 mm

Scenario:

34°C

32 mm

📌 Step 7.3.4 — Re-run Climate Prediction

Flow:

Modified Inputs

↓

ML Model

↓

New Forecast

Outputs:

🌧 Predicted Rainfall
🌡 Predicted Temperature

📌 Step 7.3.5 — GenAI Climate Explanation ⭐

After simulation:

LLM generates explanation.

Input:

forecast output

Output:

Example:

> This scenario indicates reduced rainfall and elevated temperatures, increasing drought susceptibility over the selected region.

This becomes your GenAI layer.

📌 Step 7.3.6 — Scenario Memory

Save scenarios.

Examples:

Heatwave

Weak Monsoon

Extreme Rain

Users compare:

Before

vs

After

🌍 Dashboard Flow

User:

Open Dashboard

↓

Select Region

↓

Type Scenario

↓

Generate

↓

View Climate Twin

↓

Read AI Insight

Updated Architecture

IMD + INSAT

↓

Data Pipeline

↓

Forecast Model

↓

Digital Twin

↓

GenAI Scenario Layer

↓

Dashboard

🧩 Tech Stack Addition

GenAI Layer

Model:

Gemini API
or
OpenAI API

Backend:

FastAPI

Libraries:

LangChain (optional)

Pydantic

📌 Step 7.4 — Twin State Manager

Store:

current

forecast

scenario

Structure:

state/

simulation/

📌 Step 7.5 — API Layer

Build:

GET /current

GET /forecast

POST /simulate

Backend:

FastAPI

📌 Step 7.6 — Validation

Compare:

prediction

vs

actual

Track:

📈 Accuracy
📈 Error

📌 Step 7.7 — Dashboard Connection

Flow:

React

↓

API

↓

Update UI

Dynamic updates:

✨ Fade
✨ Animation
✨ Map updates

📄 Deliverables (Phase 7)

Digital Twin

Simulation

API

Live Dashboard

You now have:

✅ Data
✅ ML
✅ Digital Twin Engine

Now comes the part judges actually see.

🟦 PHASE 8 — FRONTEND DASHBOARD & VISUALIZATION

🎯 Objective

Transform climate predictions into an interactive system that users can explore.

The dashboard should answer:

🌧️ What is happening now?
🌡️ What will happen next?
🦟 Where is it happening?
🔬 What happens if conditions change?

Duration:

🕑 **5–7 Days**

🧠 Dashboard Philosophy

Your dashboard should feel:

🟢 Scientific
🟢 Modern
🟢 Interactive

Avoid:

❌ Corporate admin panel
❌ Static charts
❌ Too many buttons

🖥️ Dashboard Architecture

User

↓

React

↓

API

↓

Digital Twin

↓

Model

📌 Step 8.1 — Create Frontend Structure

Structure:

frontend/

src/

components/

pages/

services/

hooks/

assets/

animations/

📌 Step 8.2 — Build Routing

Pages:

/

dashboard

forecast

simulation

about

📌 Step 8.3 — Design Landing Page

Hero:

MANGO
Climate Digital Twin

Sections:

🌍 Overview
🦟 Data Sources

🧠 AI Pipeline
📈 Features

Animations:

✨ Fade
✨ Slide
✨ Smooth Scroll

📌 Step 8.4 — Climate Map

Core feature.

Display:

🗺 Rainfall
🗺 Temperature

User selects:

Region

Date

Map updates instantly.

Use:

Leaflet

📌 Step 8.5 — Forecast Panel

Display:

Current

Tomorrow

3 Day

Visuals:

📈 Line Charts
📊 Trend Graphs

Use:

Plotly

📌 Step 8.6 — Simulation Controls ⭐

Build controls:

+ Temperature

− Rainfall

Reset

Output updates:

live

📌 Step 8.7 — Dashboard Components

Create reusable blocks.

Examples:

Stat Card

Map Card

Forecast Card

📌 Step 8.8 — Add Dynamic Effects

Use:

Framer Motion

Effects:

✨ Fade
✨ Reveal
✨ Hover

Add:

Lenis

for smooth scrolling.

📌 Step 8.9 — Mobile Responsiveness

Test:

📱 Mobile
💻 Desktop

📌 Step 8.10 — Loading & Error States

Examples:

Loading…

No Data

Retry

📄 Deliverables (Phase 8)

Dashboard

Map

Charts

Simulation UI

🟪 PHASE 9 — SYSTEM INTEGRATION & TESTING

🎯 Objective

Connect every module into one working application.

Duration:

🕒 **3–5 Days**

🧠 Goal

Before:

Frontend

Backend

ML

After:

One Product

🏗️ Final Architecture

Frontend

↓

Backend

↓

Digital Twin

↓

Model

↓

Dataset

📌 Step 9.1 — Connect Frontend → Backend

Frontend calls:

/current

/forecast

/simulate

📌 Step 9.2 — Connect Backend → ML

Flow:

Input

↓

Model

↓

Prediction

📌 Step 9.3 — Create Service Layer

Structure:

services/

forecast.js

simulation.js

📌 Step 9.4 — API Validation

Test:

valid input

invalid input

empty

📌 Step 9.5 — Performance Testing

Measure:

⏱️ API Time
🧠 Prediction Time

Target:

<2 seconds

📌 Step 9.6 — Integration Testing

Test full journey:

Open Dashboard

↓

Select Region

↓

Forecast

↓

Simulate

📌 Step 9.7 — Error Recovery

Handle:

⚠ API down
⚠ Missing data
⚠ Model failure

Show:

Fallback UI

📌 Step 9.8 — Final Folder Structure

backend/

frontend/

ml/

data/

docs/

📌 Step 9.9 — Version Freeze

Create:

release-v1

Tag:

git tag v1.0

📄 Deliverables (Phase 9)

Working System

Connected APIs

Stable Demo

🟧 PHASE 10 — FINAL PRESENTATION & HACKATHON SUBMISSION

🎯 Objective

Convert engineering work into a convincing story.

Duration:

🕒 **2–3 Days**

🧠 Presentation Formula

Problem

↓

Solution

↓

Demo

↓

Impact

📌 Step 10.1 — Create PPT

Slides:

Title

Problem

Architecture

Data

ML

Dashboard

Results

Future

📌 Step 10.2 — Create Architecture Diagram

Show:

Data

↓

AI

↓

Twin

↓

Dashboard

📌 Step 10.3 — Prepare Demo

Flow:

Open Site

↓

Choose Region

↓

Show Current

↓

Forecast

↓

Simulate

📌 Step 10.4 — Record Backup Video

Record:

🎥 Full demo
🎥 3–5 minutes

Include:

all features

📌 Step 10.5 — Results Slide

Include:

📈 MAE
📈 RMSE
📈 Accuracy

Add:

Screenshots.

📌 Step 10.6 — Explain Innovation ⭐

Explain:

Digital Twin

+

AI

+

National Data

📌 Step 10.7 — Future Scope

Ideas:

🛰️ More satellite inputs
🌾 Agriculture insights
🌊 Flood alerts
🌡️ Long-term simulation

📌 Step 10.8 — Submission Checklist

Checklist:

Code

PPT

Video

README

Demo

📄 Deliverables (Phase 10)

Submission Package

Presentation

Demo

🏁 PROJECT COMPLETE

Final Flow:

Data

↓

Processing

↓

ML

↓

Digital Twin

↓

API

↓

Dashboard

↓

Submission